

Lab Exercise 2: Matrix Multiplication & Exception Handling

Objective: To write a Java program that multiplies two dimensions-compatible matrices and securely catches dimensional mismatch errors using custom exceptions.

Aim: To implement a robust, encapsulated matrix multiplication routine that demonstrates standard 2D arrays, input verification, and structural exception safeguards.

Theory: Matrix multiplication requires the number of columns in Matrix A to exactly equal the number of rows in Matrix B. Brute-force calculation executes with a time complexity of $O(n^3)$. Throwing a targeted custom exception prevents illegal memory or indexing exceptions before nested loops are initiated.

Algorithm / Procedure:

1. Input: Rows and columns of Matrix A, followed by its elements. Then rows, columns, and elements of Matrix B.
2. Verify if $\text{columnsA} == \text{rowsB}$. If not, throw a custom exception `MatrixMismatchException`.
3. Initialize an output tracking matrix result of size $\text{rowsA} \times \text{columnsB}$.
4. Apply three nested processing loops to compute the cell-by-cell dot products.
5. Print the calculated matrix.

Program Code: `Problem2_MatrixMultiplication.java`

```
import java.util.*;

class MatrixMismatchException extends Exception {

    public MatrixMismatchException(String message) { super(message); }

}

public class Problem2_MatrixMultiplication {

    public static int[][] multiplyMatrices(int[][] A, int[][] B) throws MatrixMismatchException {

        // TODO: Implement your matrix multiplication and validation logic here

        return new int[0][0]; // placeholder

    }
```

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    try {
        System.out.print("Enter rows and cols for Matrix A: ");
        int rA = sc.nextInt(); int cA = sc.nextInt();
        int[][] A = new int[rA][cA];
        for(int i=0; i<rA; i++) for(int j=0; j<cA; j++) A[i][j] = sc.nextInt();

        System.out.print("Enter rows and cols for Matrix B: ");
        int rB = sc.nextInt(); int cB = sc.nextInt();
        int[][] B = new int[rB][cB];
        for(int i=0; i<rB; i++) for(int j=0; j<cB; j++) B[i][j] = sc.nextInt();

        int[][] result = multiplyMatrices(A, B);
        System.out.println("Resulting Matrix:");
        for(int[] row : result) System.out.println(Arrays.toString(row));
    } catch (MatrixMismatchException e) {
        System.out.println("Error: " + e.getMessage());
    } finally { sc.close(); }
}
}

```

Sample Input / Output:

Input:

2 3

1 2 3

4 5 6

3 2

7 8

9 1

2 3

Output:

Resulting Matrix:

[31, 19]

[85, 55]

Evaluation Criteria (Total Marks: 10):

- Program compiles and runs successfully (2 Marks)
- Handling custom dimension validations via Exception blocks (3 Marks)
- Correct matrix multiplication output data indices (2 Marks)
- Formatted array layout rendering handling (2 Marks)
- Indentation and code presentation formatting (1 Mark)